



JAVA INTERVIEW SERIES:

Collection Framework

@JITHENDAR REDDY



1.Explain the collection Hierarchy in java ?

The Java Collection Framework provides interfaces like List, Set, Queue, and Map to handle groups of objects.

- List → Ordered, allows duplicates
- Set → Unordered, no duplicates
- Queue → FIFO structure
- Map → Key-value pairs (unique keys)

The hierarchy starts from Iterable → Collection → List/Set/Queue, and Map forms a separate branch.



2. Why does Map not extend the Collection interface in Java?

The Map interface does not extend the Collection interface because they represent different data structures.

A Collection stores a group of individual elements, while a Map stores key–value pairs.

If Map extended Collection, it would inherit methods like `add()` or `iterator()` that don't logically apply to key–value mappings.

Therefore, Map is kept as a separate branch in the Java Collection Framework.



3. Difference between fail-fast and fail-safe Iterators?

A Fail-Fast Iterator immediately throws a `ConcurrentModificationException` if the collection is modified while iterating, because it works directly on the original collection.

A Fail-Safe Iterator, on the other hand, works on a clone of the collection and therefore doesn't throw exceptions even when the collection is modified concurrently.

Examples:

- Fail-Fast: `ArrayList`, `HashMap`
- Fail-Safe: `CopyOnWriteArrayList`, `ConcurrentHashMap`



4.what do you understand by Blockingqueue?

A BlockingQueue in Java is a thread-safe queue that is used to handle elements in producer-consumer scenarios, where one thread is producing data and another is consuming it. It belongs to the `java.util.concurrent` package and supports automatic blocking of threads:

- If a thread tries to insert an element into a full queue, it waits (blocks) until space becomes available.
- If a thread tries to remove an element from an empty queue, it waits (blocks) until an element is available.

The BlockingQueue handles synchronization internally — so we don't need to use `wait()` or `notify()` manually.



5.what is difference Synchronized collection and concurrent collection?

A Synchronized Collection achieves thread safety by locking the entire collection, allowing only one thread to access it at a time — for example, Vector or Collections.synchronizedList().

A Concurrent Collection, introduced in java.util.concurrent, allows multiple threads to operate safely and efficiently using fine-grained or lock-free mechanisms — for example, ConcurrentHashMap or CopyOnWriteArrayList.

In short:

- Synchronized = Simple but slower (single lock, fail-fast)
- Concurrent = Efficient and scalable (fine-grained locks, fail-safe)



6.internal working of hashmap

Internally, a HashMap stores key–value pairs in an array of nodes (buckets).

Each node stores a key, value, hash, and next pointer.

The key's hashCode() is used to determine the bucket index using $(n - 1) \& \text{hash}$.

When you call put(), it stores the entry in the corresponding bucket; if another key maps to the same bucket, a collision occurs, and entries are stored in a linked list (or red-black tree in Java 8 for performance).

The get() method uses the same hash function to find the bucket and retrieves the value by matching the key.

Default capacity = 16, Load factor = 0.75, and time complexity = $O(1)$ in average cases.

This design gives HashMap its speed and efficiency for lookups and insertions.

